

LAPORAN PROYEK AKHIR KRIPTOGRAFI

CRYPTCHAT: PESAN END-TO-END ENCRYPTED

Disusun untuk memenuhi project akhir Mata Kuliah Kriptografi

Dosen Pengampu: Lia Farhatuaini, S.Kom, M.Cs.



Disusun Oleh:

Salwa Ayudia Putri (2488010042)

Dzaky Muhammad Nafis (2488010048)

Asfahani Hasyim (2488010070)

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU TARBIYAH DAN KEGURUAN

UIN SIBER SYEKH NURJATI CIREBON

2026 / 1447

A. Pendahuluan

1. Latar Belakang

Di era digital seperti sekarang ini, komunikasi melalui internet telah menjadi kebutuhan utama masyarakat. Berbagai aplikasi pesan instan seperti WhatsApp, Telegram, dan Signal digunakan setiap hari untuk mengirim informasi pribadi, dokumen penting, maupun percakapan bisnis. Namun, semakin meningkatnya penggunaan layanan digital, semakin meningkat juga risiko kebocoran data, penyadapan, dan akses ilegal terhadap isi pesan.

Banyak sistem aplikasi pesan instan yang masih menyimpan data dalam bentuk plaintext di server, sehingga administrator server maupun pihak ketiga berpotensi membaca isi komunikasi pengguna. Hal ini menjadi ancaman serius terhadap privasi dan keamanan data.

Untuk mengatasi masalah tersebut, digunakan konsep End-to-End Encryption (E2EE), yaitu mekanisme di mana hanya pengirim dan penerima yang dapat membaca isi pesan. Server hanya berfungsi sebagai perantara pengiriman data tanpa mengetahui isi pesan sebenarnya.

Berdasarkan permasalahan tersebut, kelompok kami merancang sebuah aplikasi Cryptchat, yaitu sistem pesan berbasis web yang menerapkan enkripsi end-to-end menggunakan algoritma kriptografi modern.

2. Tujuan Sistem

Pembuatan aplikasi Cryptchat ini bertujuan untuk:

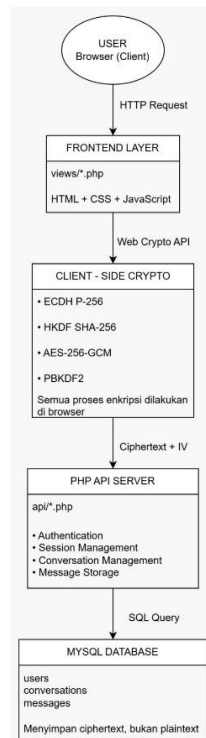
- 1) Membuat aplikasi chat berbasis web yang aman di mana isi pesan hanya bisa dibaca oleh pengirim dan penerima dan tidak bisa dibaca oleh server.
- 2) Menerapkan konsep End-to-End Encryption pada sistem pesan.
- 3) Mengimplementasikan pertukaran kunci (key exchange) yang aman menggunakan ECDH.

3. Pengguna yang Dituju

Sistem Cryptchat ini dirancang untuk semua pengguna dari semua kalangan yang membutuhkan komunikasi aman.

B. Arsitektur Sistem

1. Diagram Arsitektur



2. Komponen-komponen sistem

1) Client (Browser User)

Berfungsi untuk proses login dan register user, generate ECDH keypair (public key dan private key), melakukan PBKDF2 key derivation dari password user, mengenkripsi private key dan pesan menggunakan AES-256-GCM, membuat shared secret menggunakan ECDH, melakukan derivasi session menggunakan HKDF, mendekripsi private key dan pesan yang diterima, dan menampilkan isi chat kepada user.

2) Server PHP (API)

Digunakan untuk mengatur route dan endpoint aplikasi, memproses login dan register, memvalidasi password user, mengelola session login, melakukan hashing password user, menyimpan ciphertext pesan ke database, menyimpan public key user, menyimpan encrypted private key user, serta menghubungkan aplikasi dengan database MySQL

3) Database (MySQL)

Berfungsi untuk menyimpan data akun user (username, id), password hash user, public key user, encrypted private key user, salt dan IV enkripsi private key, ciphertext pesan, IV pesan, conversation id, fingerprint, timestamp pesan.

3. Alur Data

➤ Saat Register

- 1) User mengisi username dan password
- 2) Browser membuat ECDH key pair menggunakan Web Crypto API
- 3) Browser membuat random salt
- 4) Password diproses menggunakan PBKDF2 untuk menghasilkan AES-256 key
- 5) Private key di enkripsi menggunakan AES-256-GCM
- 6) Browser mengirim username, password, public key, encrypted private key, private key salt, private key IV ke backend PHP
- 7) Server melakukan hash password menggunakan password_hash()
- 8) Data disimpan ke database

➤ Saat Login

- 1) User memasukkan username dan password
- 2) Server memverifikasi password hash
- 3) Jika cocok, encrypted private key, private key salt, private key IV dikirim ke browser
- 4) Browser melakukan PBKDF2 kembali menggunakan password user agar menghasilkan AES-256 key yang sama seperti saat register
- 5) Browser mendekripsi private key menggunakan AES-256-GCM
- 6) Setelah berhasil mendekripsikan private key, password akan langsung dihapus dari sessionStorage
- 7) Private key disimpan sementara di sessionStorage selama sesi berlangsung

➤ Saat Mengirim Pesan

- 1) Pengirim mengambil public key penerima
- 2) Browser menghitung shared secret dengan ECDH
- 3) Shared secret diproses melalui HKDF untuk menghasilkan session key
- 4) Pesan dienkripsi dengan AES-256-GCM
- 5) Ciphertext dan IV pesan dikirim ke server
- 6) Server menyimpan ciphertext ke database

➤ Saat Menerima Pesan

- 1) Ciphertext diambil dari database melalui server
- 2) Browser penerima mengambil public key pengirim
- 3) Browser menghitung shared secret menggunakan ECDH
- 4) Shared secret diproses kembali menggunakan HKDF
- 5) Ciphertext didekripsi menggunakan AES-256-GCM
- 6) Isi pesan asli ditampilkan kepada user

C. Desain Kriptografi

Komponen	Algoritma	Parameter	Lokasi Eksekusi
Key Exchange	ECDH P-256	Elliptic Curve P-256	Client (Browser)
Key Derivation	PBKDF2-SHA256	100.000 Iterasi	Client (Browser)
Session Key Derivation	HKDF-SHA256	Session key	Client (Browser)
Enkripsi Pesan	AES-256-GCM	IV 12 byte	Client (Browser)
Password Hashing	password_hash()	bcrypt	Sever (PHP)
Random Generator	crypto.getRandomValues()	CSPRNG	Client (Browser)

1. ECDH

- Algoritma yang dipilih

Cryptchat menggunakan ECDH dengan kurva P-256 untuk proses key exchange antar user. ECDH digunakan untuk menghasilkan shared secret tanpa mengirim secret tersebut melalui jaringan.

- Alasan pemilihan

ECDH dipilih karena mendukung secure key exchange, lebih ringan disbanding RSA, ukuran key lebih kecil, performa cepat, aman untuk aplikasi modern, dan cocok untuk digunakan komunikasi real-time.

- Alternatif yang dipertimbangkan

Alternatif yang dipertimbangkan adalah RSA. RSA tidak dipilih karena ukuran key lebih besar, proses komputasi lebih berat, dan kurang efisien untuk aplikasi real-time.

2. PBKDF2 + SHA 256

- Algoritma yang dipilih

Cryptchat menggunakan PBKDF2 dengan SHA-256 untuk mengubah password menjadi AES key.

- Alasan pemilihan

PBKDF2 dipilih karena mendukung salt, mendukung iteration count, tersedia pada Web Crypto API, memperlambat brute force attack, dan aman untuk password-based key derivation.

- Alternatif yang dipertimbangkan

Alternatif yang dipertimbangkan adalah bcrypt dan Argon2. Argon2 lebih modern dan tahan GPU attack, tetapi belum memiliki dukungan native penuh pada Web Crypto API.

3. HKDF

- Algoritma yang dipilih

HKDF digunakan untuk membentuk session key dari shared secret hasil ECDH.

- Alasan pemilihan

HKDF dipilih karena aman untuk key derivation modern, cocok digunakan Bersama ECDH, menghasilkan key yang konsisten, dan memisahkan shared secret dari key akhir.

- Alternatif yang dipertimbangkan

Alternatif yang dipertimbangkan adalah menggunakan shared secret langsung sebagai AES key. Namun metode tersebut tidak dipilih karena kurang fleksibel, tidak melakukan key normalization, dan kurang aman dibanding HKDF.

4. AES-256-GCM

- Algoritma yang dipilih

AES-256-GCM digunakan untuk mengenkripsi private key dan mengenkripsi isi pesan.

- Alasan pemilihan

AES-256-GCM dipilih karena cepat, aman, mendukung authenticated encryption, memiliki integrity protection, dan mampu mendeteksi modifikasi ciphertext.

- Alternatif yang dipertimbangkan

Alternatif yang dipertimbangkan adalah DES, Triple DES, dan AES-CBC. DES tidak dipilih karena sudah tidak aman. AES-CBC tidak dipilih karena tidak memiliki authentication tag dan rentan padding oracle attack.

D. Implementasi

- **Generate ECDH Key Pair (Registrasi)**

```
async function generateKeyPair() {
  try {
    const keyPair = await crypto.subtle.generateKey(
      {
        name: "ECDH",
        namedCurve: "P-256",
      },
      true,
      ["deriveKey", "deriveBits"],
    );

    const publicKey = await crypto.subtle.exportKey("spki", keyPair.publicKey);

    const privateKey = await crypto.subtle.exportKey(
      "pkcs8",
      keyPair.privateKey,
    );

    return {
      publicKey: arrayBufferToBase64(publicKey),
      privateKey: arrayBufferToBase64(privateKey),
    };
  } catch (error) {
    console.error(error);
  }
}
```

Penjelasan:

Kurva P-256 dipilih karena didukung Web Crypto API di semua browser modern dan menyediakan keamanan 128-bit. Seluruh proses terjadi di browser. Private key tidak pernah dikirim ke server dalam bentuk plaintext

- **Enkripsi Private Key (PBKDF2 + AES-256-GCM)**

```
async function encryptPrivateKey(privateKey, password) {
  const salt = crypto.getRandomValues(new Uint8Array(16));

  const key = await derivePasswordKey(password, arrayBufferToBase64(salt));

  const iv = crypto.getRandomValues(new Uint8Array(12));

  const encrypted = await crypto.subtle.encrypt(
    {
      name: "AES-GCM",
      iv,
    },
    key,
    new TextEncoder().encode(privateKey),
  );

  return {
    ciphertext: arrayBufferToBase64(encrypted),
    iv: arrayBufferToBase64(iv),
    salt: arrayBufferToBase64(salt),
  };
}
```

Penjelasan:

PBKDF2 dengan 100.000 iterasi digunakan sebagai KDF agar brute force password menjadi sangat lambat. Salt 128-bit acak (dibangkitkan menggunakan `crypto.getRandomValues()`) per user memastikan dua user dengan password sama tetap menghasilkan kunci enkripsi berbeda. AES-256-GCM dipilih karena menyediakann enkripsi sekaligus autentikasi integritas.

- Dekripsi Private Key Saat Login

```
async function decryptPrivateKey(ciphertext, iv, password, salt) {
  try {
    if (!ciphertext || !iv || !password || !salt) {
      throw new Error("Data dekripsi private key tidak lengkap.");
    }

    const key = await derivePasswordKey(password, salt);

    const decrypted = await crypto.subtle.decrypt(
      {
        name: "AES-GCM",
        iv: base64ToArrayBuffer(iv),
      },
      key,
      base64ToArrayBuffer(ciphertext),
    );

    const privateKey = new TextDecoder().decode(decrypted);

    if (!privateKey || privateKey.trim() === "") {
      throw new Error("Private key hasil dekripsi kosong.");
    }

    return privateKey;
  } catch (error) {
    console.error("Gagal mendekripsi private key:", error);
    return null;
  }
}
```

Penjelasan:

Try-catch ditambahkan agar kegagalan didekripsi (password salah) menghasilkan return null yang aman, bukan exception tidak tertangkap. loginPassword dihapus dari sessionStorage segera setelah private key berhasil di dekripsi untuk meminimalkan eksposur password di memori browser. Private key hanya disimpan sementara selama sesi login berlangsung.

- ECDH Key Exchange dan Derivasi Session Key (HKDF)

```
async function computeSharedSecret(privateKey, publicKey) {
  return crypto.subtle.deriveBits(
    {
      name: "ECDH",
      public: publicKey,
    },
    privateKey,
    256,
  );
}

async function createSessionKey(targetUserId, conversationId) {
  const response = await fetch(
    `../api/get_public_key.php?user_id=${targetUserId}`,
  );
  const data = await response.json();

  if (!data.success) {
    throw new Error("Public key target tidak ditemukan");
  }

  const privateKeyBase64 = sessionStorage.getItem("privateKey");

  if (!privateKeyBase64) {
    throw new Error("Private key tidak ditemukan");
  }

  const privateKey = await importPrivateKey(privateKeyBase64);
  const publicKey = await importPublicKey(data.public_key);

  const sharedSecret = await computeSharedSecret(privateKey, publicKey);

  return await deriveSessionKey(sharedSecret, conversationId);
}
```

```
async function deriveSessionKey(sharedSecret, conversationId) {
  // Import raw shared bits ke HSM
  const importedKey = await crypto.subtle.importKey(
    "raw",
    sharedSecret,
    "HKDF",
    false,
    ["deriveKey"],
  );

  // derive AES key
  return await crypto.subtle.deriveKey(
    {
      name: "HKDF",
      hash: "SHA-256",
      salt: new TextEncoder().encode("cryptchat-" + conversationId),
      info: new TextEncoder().encode("session-key"),
    },
    importedKey,
    {
      name: "AES-GCM",
      length: 256,
    },
    true,
    ["encrypt", "decrypt"],
  );
}
```

Penjelasan:

Shared secret ECDH tidak langsung dipakai sebagai kunci AES karena output ECDH tidak terdistribusi uniform, HKDF-SHA256 digunakan untuk mengekstraknya menjadi kunci yang kuat. Conversation ID sebagai salt HKDF memastikan setiap percakapan menghasilkan session key yang berbeda meski melibatkan user yang sama.

- **Enkripsi dan Dekripsi Pesan (AES-256-GCM)**

```
async function encryptMessage(message, sessionKey) {
  const iv = crypto.getRandomValues(new Uint8Array(12));

  const encrypted = await crypto.subtle.encrypt(
    {
      name: "AES-GCM",
      iv,
    },
    sessionKey,
    new TextEncoder().encode(message),
  );

  return {
    ciphertext: arrayBufferToBase64(encrypted),
    iv: arrayBufferToBase64(iv),
  };
}

async function decryptMessage(ciphertext, iv, sessionKey) {
  try {
    const decrypted = await crypto.subtle.decrypt(
      {
        name: "AES-GCM",
        iv: base64ToArrayBuffer(iv),
      },
      sessionKey,
      base64ToArrayBuffer(ciphertext),
    );

    return new TextDecoder().decode(decrypted);
  } catch {
    return "[Decrypt Failed]";
  }
}
```

Penjelasan:

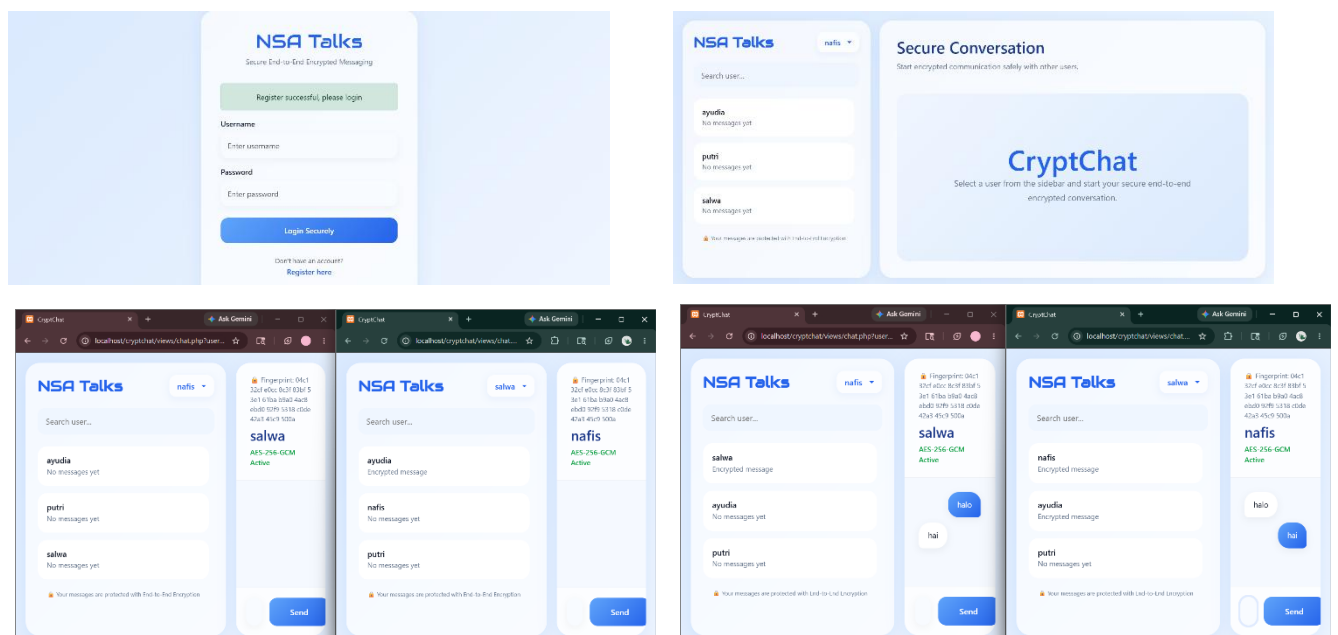
IV acak 96-bit dibangkitkan ulang setiap pesan. IV disimpan bersama ciphertext karena tidak bersifat rahasia, namun wajib unik per pesan untuk mencegah reuse attack pada AES-GCM.

E. Pengujian

1) Skenario Normal

No	Skenario	Langkah Uji	Hasil yang Diharapkan
1.	Registrasi User Baru	Jalankan register.php di browser, isi username dan password, klik register	Data masuk ke tabel users, alert registrasi berhasil, diarahkan ke login.php
2.	Login Benar	Masukkan username dan password	Session PHP dibuat, private key berhasil di dekripsi, user masuk ke dashboard
3.	Membuat conversation dan kirim pesan	Pilih user lain, halaman chat terbuka, ketik pesan, klik send	Chipertext dan IV tersimpan di tabel messages, pesan tampil di chat pengirim
4.	Menerima pesan	Login user kedua, buka conversation, pesan diterima	Cipertext diambil, di dekripsi, pesan asli tampil di chat
5.	Fingerprint session key	Buka conversation	Fingerprint session key tampil sama di kedua client

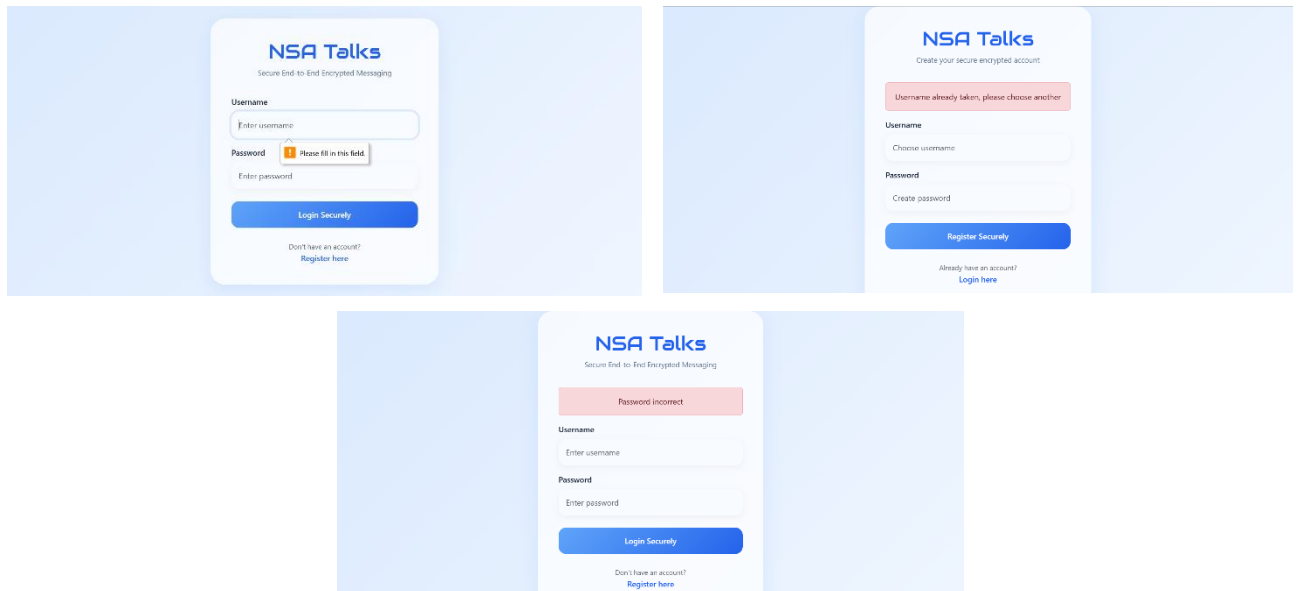
Screenshoot Hasil Pengujian:



2) Skenario Error

No	Skenario	Pemicu	Hasil yang Diharapkan/Temuan
1.	Field login kosong	Username atau password tidak diisi	Sistem menampilkan alert bahwa field wajib diisi
2.	Username sudah digunakan saat register	Registrasi menggunakan username yang sudah ada	Server menolak registrasi dan menampilkan pesan error
3.	Password login salah	User memasukkan password yang salah	Login gagal dan session tidak dibuat

Screenshoot Hasil Pengujian:

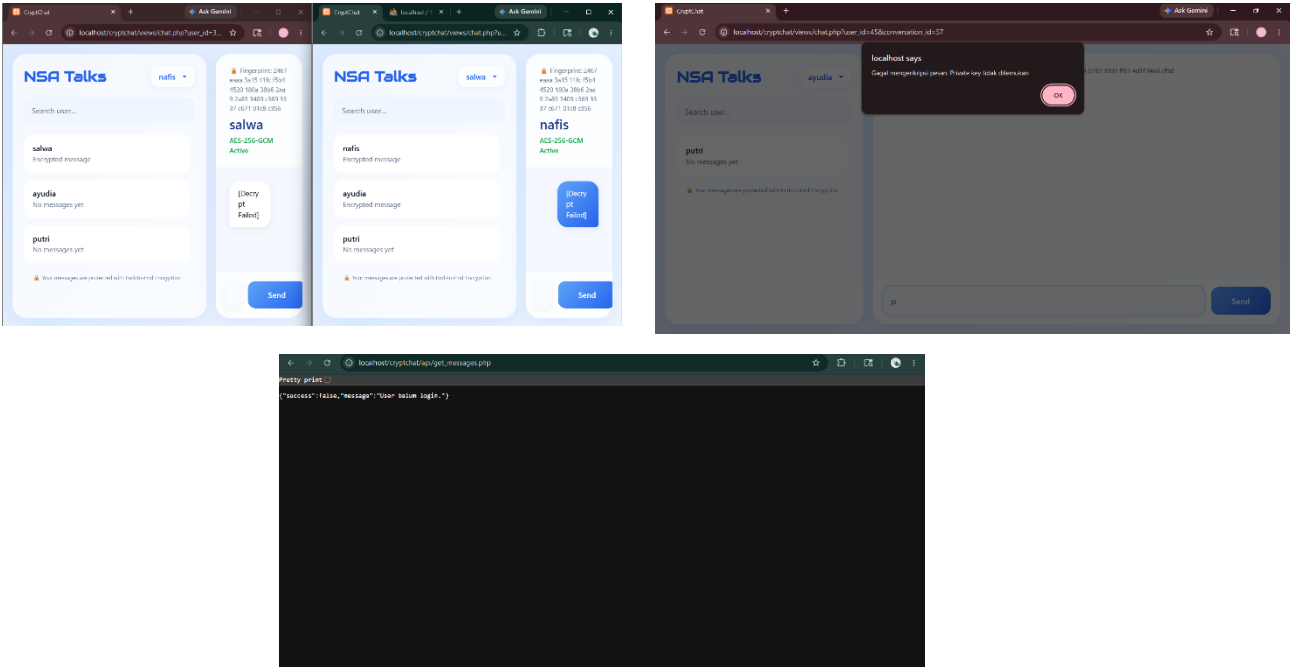


3) Skenario Serangan

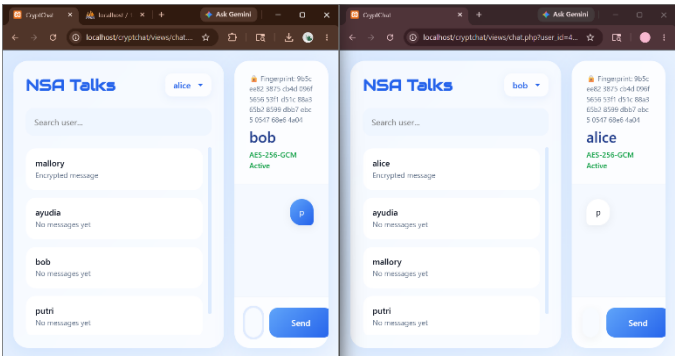
No	Skenario	Cara Uji	Hasil yang Diharapkan/Temuan
1.	Tempering IV	Modifikasi IV manual di database	Decrypt gagal karena authentication tag tidak valid
2.	Private key hilang	Hapus privateKey yang ada di sessionStorage	Browser gagal untuk mengenkripsi pesan

3.	Tempering ciphertext	Mengubah beberapa karakter ciphertext di database	Decrypt gagal dan pesan tidak dapat dibaca
4.	Akses API messages tanpa login	Fetch api/get_messages.php dari browser tanpa login	Tampil message “User belum login”
5.	MITM Sederhana (Public key substitution attack)	Mengganti public key milik Bob dengan public key user lain	Fingerprint disisi Alice berubah dan decrypt gagal disisi bob

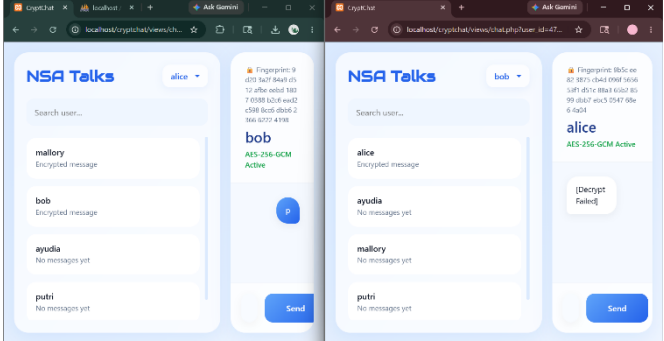
Screenshoot Hasil Pengujian:



Sebelum serangan:



Sesudah serangan:



F. Keterbatasan & Refleksi

1. Private key tersimpan di sessionStorage dalam kondisi sudah terdekripsi selama sesi tersebut berlangsung. Jika terjadi XSS pada halaman yang aktif, attacker dapat membaca private key dan menggunakannya untuk compute shared secret serta mendekripsi pesan pada percakapan yang sedang dibuka.
 - **Perbaikan kedepannya:** Private key sebaiknya disimpan menggunakan Web Crypto API dengan 'extractable: false' dan IndexedDB agar tidak mudah diambil oleh JavaScript maupun serangan XSS.
2. Sistem tidak memiliki forward secrecy. Jadi selama private key user sama, session key yang dihasilkan selalu sama untuk percakapan yang sama. Jika private key bocor di masa depan, semua pesan lama yang tersimpan di server dapat dididekripsi.
 - **Perbaikan kedepannya:** Sistem dapat dikembangkan dengan menerapkan Double Ratchet Algorithm agar session key terus berubah secara otomatis. Dengan begitu, jika satu key bocor, pesan lain tetap aman dan tidak ikut terbuka.
3. Verifikasi identitas terhadap serangan Man-in-the-Middle hanya bersifat manual melalui fingerprint session key yang ditampilkan di header chat. Sistem tidak memiliki mekanisme otomatis untuk mendeteksi jika public key user diganti oleh pihak lain di server.
 - **Perbaikan kedepannya:** Menambahkan mekanisme Trust on First Use (TOFU) yang menyimpan hash public key saat pertama kali berkomunikasi dengan user tertentu, dan secara otomatis memperingatkan user jika public key tersebut berubah di kemudian hari.
4. Aplikasi tidak memiliki fitur real-time messaging. User harus me-refresh halaman secara manual untuk melihat pesan baru yang masuk, sehingga pengalaman penggunaan tidak seperti aplikasi chat modern pada umumnya.
 - **Perbaikan kedepannya:** Mengimplementasikan WebSocket atau Server-Sent Events (SSE) untuk push notification pesan baru ke browser tanpa perlu refresh.
5. Sistem hanya mendukung pengiriman pesan teks. Tidak ada fitur pengiriman gambar, file, atau media lainnya yang juga dienkripsi end-to-end.
 - **Perbaikan kedepannya:** mengenkripsi file menggunakan AES-256-GCM dengan session key yang sama, kemudian mengirimkan binary blob terenkripsi ke server. Dengan ini, file dapat dikirim dengan jaminan kerahasiaan yang setara dengan pesan teks.